

Documentação da Bibliografia de Análise e Projeto Orientados a Objetos (APOO)

Este documento apresenta uma análise detalhada de cada livro que compõe a bibliografia oficial da disciplina de **Análise e Projeto Orientados a Objetos (APOO)** da instituição **IFSP Câmpus São Carlos**.

O objetivo desta documentação é resumir a proposta de cada obra, destacar seus conceitos fundamentais e estabelecer a correlação direta de cada um com o conteúdo programático da disciplina e com a estrutura exigida para a **Avaliação Prática** (desenvolvimento de arquitetura em Python).

Índice Analítico

1. Architecture Patterns with Python (Harry Percival & Bob Gregory)
2. Orientação a Objetos e SOLID para Ninjas (Maurício Aniche)
3. UML Essencial (Martin Fowler)
4. Análise e Modelagem de Sistemas com a UML (Luiz Antônio de Moraes Pereira)
5. UML, Metodologias e Ferramentas CASE (Alberto M. R. da Silva & Carlos A. E. Videira)
6. Python e Orientação a Objetos (Curso PY-14) (Caelum)
7. Tabela de Mapeamento de Conceitos e o Projeto Prático

1. Architecture Patterns with Python

Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices

- **Autores:** Harry Percival & Bob Gregory
- **Ano:** 2020
- **Editora:** O'Reilly Media

[!IMPORTANT]

Relevância Crítica para a Avaliação Prática:

Este livro é a base arquitetural exata para o projeto prático sorteado em dupla (Semana 17 e 18). Ele descreve detalhadamente o funcionamento de cada arquivo fornecido (`bootstrap.py`, `messagebus.py`, `unit_of_work.py`) e dos arquivos que devem ser implementados (`domain.py`, `messages.py`, `handlers.py`, `repository.py`, `read_model.py`).

Visão Geral

A obra foca em como construir aplicações robustas, flexíveis e altamente testáveis em Python, aplicando padrões clássicos de arquitetura de software (como Portas e Adaptadores/Hexagonal) acoplados ao **Domain-Driven Design (DDD)**. Os autores demonstram que, ao desacoplar a lógica de negócios central da infraestrutura (como banco de dados e APIs), o sistema torna-se infinitamente mais fácil de evoluir e testar.

Conceitos-Chave

- **Domain-Driven Design (DDD):**
- **Domain Model:** O núcleo da aplicação contendo as regras e termos de negócio. Deve ser escrito usando uma linguagem ubíqua (comum entre desenvolvedores e especialistas do negócio).
- **Entities:** Objetos com identidade própria e duradoura (ex: um ID único).
- **Value Objects:** Objetos definidos apenas por seus valores e imutáveis (ex: um endereço ou quantidade). Em Python, são frequentemente implementados com `@dataclass(frozen=True)`.
- **Aggregates:** Um grupo de objetos associados que tratamos como uma unidade única de consistência de dados.
- **Padrões Estruturais de Desacoplamento:**
- **Repository Pattern:** Abstração sobre o armazenamento de persistência de dados. Faz com que a persistência se comporte como se fosse uma coleção em memória (encapsulando as operações SQL de banco de dados).
- **Unit of Work (UOW):** Garante a atomicidade das transações de negócios. Ele rastreia alterações e garante que todas as operações em um caso de uso sejam aplicadas juntas no banco de dados, ou revertidas em caso de falha.
- **Service Layer (Handlers):** A camada que orquestra os casos de uso. Recebe comandos, interage com o Domain Model e persiste modificações via UOW.
- **Arquitetura Baseada em Mensagens (Event-Driven):**
- **Domain Events:** Eventos que representam fatos que já aconteceram no sistema (ex: **ProductAllocated**).
- **Message Bus:** Canal que despacha comandos e eventos para seus respectivos manipuladores (handlers), mantendo as partes do sistema desacopladas.

2. Orientação a Objetos e SOLID para Ninjas

Projetando classes flexíveis

- **Autor:** Mauricio Aniche
- **Ano:** 2015
- **Editora:** Casa do Código

[!TIP]

Relevância para a Disciplina:

Essencial para as fases iniciais de modelagem do domínio (**domain.py**) e para responder às perguntas conceituais da arguição prática (~50% da nota). O livro auxilia a compreender conceitos cruciais como acoplamento, coesão e encapsulamento em nível prático.

Visão Geral

Este livro tem como propósito ajudar o desenvolvedor a dar o salto do paradigma procedural disfarçado de classes para a verdadeira Orientação a Objetos. O autor ensina a projetar classes altamente coesas e fracamente acopladas aplicando os princípios SOLID por meio de cenários de refatoração de código.

Conceitos-Chave

- **Coesão e o Single Responsibility Principle (SRP):** Uma classe deve ter um único propósito ou uma única razão para mudar. Classes não coesas acumulam múltiplos papéis e tornam-se difíceis de manter.
- **Acoplamento e o Dependency Inversion Principle (DIP):** Módulos de alto nível não devem depender de módulos de baixo nível (como o banco de dados diretamente), mas de abstrações (interfaces/classes abstratas). Reduzir o acoplamento impede que uma alteração em uma classe quebre várias outras.
- **Classes Abertas e o Open/Closed Principle (OCP):** Entidades de software devem estar abertas para extensão, mas fechadas para modificação. Isso é comumente atingido por meio do polimorfismo.
- **Princípio de Substituição de Liskov (LSP):** Subclasses devem ser capazes de substituir suas superclasses sem corromper o comportamento esperado do sistema.
- **Princípio da Segregação de Interfaces (ISP):** Clientes não devem ser forçados a depender de métodos que não utilizam (interfaces mais específicas são preferíveis a interfaces genéricas e pesadas).

3. UML Essencial

Um breve guia para a linguagem-padrão de modelagem de objetos

- **Autor:** Martin Fowler
- **Ano:** 2007 (3ª edição)
- **Editora:** Bookman

[!NOTE]

Relevância para a Disciplina:

Referência conceitual padrão para a criação dos diagramas exigidos nas avaliações teóricas e práticas (UML partes 1 a 4). Explica a filosofia do uso "ágil" da UML.

Visão Geral

Martin Fowler apresenta um guia conciso e muito prático sobre a UML, focando nos diagramas e conceitos mais utilizados no mundo real do desenvolvimento de software. Ele desmistifica a UML como um processo pesado e burocrático e a propõe como um facilitador de comunicação.

Conceitos-Chave

- **Perspectivas de Uso da UML:**
- **UML como Esboço (Sketch):** Uso informal no quadro branco para comunicar ideias de design rapidamente.
- **UML como Projeto (Blueprint):** Projetos detalhados criados com ferramentas CASE com o intuito de gerar código de forma sistemática.

- **UML como Linguagem de Programação:** Geração de código executável diretamente a partir do modelo UML.
- **Diagramas Essenciais:**
 - **Diagrama de Classes:** Representação estrutural contendo classes, atributos, operações e seus relacionamentos (associação, agregação, composição, herança).
 - **Diagrama de Sequência:** Representação dinâmica que mostra como os objetos interagem ao longo do tempo para realizar um caso de uso específico.
 - **Diagrama de Casos de Uso:** Visão geral do comportamento e dos atores envolvidos com o sistema.

4. Análise e Modelagem de Sistemas com a UML

com dicas e exercícios resolvidos

- **Autor:** Luiz Antônio de Moraes Pereira
- **Ano:** 2011 (1ª edição)
- **Editora:** Publicação Própria (PUC-Rio)

Visão Geral

Este livro visa preencher a lacuna entre a extensa documentação oficial da UML e a prática diária de engenharia de requisitos e análise de sistemas. Com foco didático acentuado, a obra condensa as estruturas essenciais da UML em formato de perguntas, respostas, boas práticas e exercícios contextualizados.

Conceitos-Chave

- **Mapeamento de Regras de Negócio para Artefatos UML:** Técnicas para ler requisitos textuais descritos por clientes e transformá-los de forma precisa em diagramas de caso de uso e classes.
- **Modelagem Estrutural vs. Comportamental:** Diferenciação clara e boas práticas para modelar a estrutura estática das classes (atributos, tipos, multiplicidades) em paralelo com o comportamento (sequenciamento de mensagens e transição de estados de objetos).
- **Exercícios Práticos de Fixação:** Ampla lista de cenários reais resolvidos passo a passo, detalhando o porquê de cada decisão de modelagem gráfica.

5. UML, Metodologias e Ferramentas CASE

Linguagem de Modelação UML, Metodologias e Ferramentas CASE na Concepção e Desenvolvimento de Software

- **Autores:** Alberto Manuel Rodrigues da Silva & Carlos Alberto Escaleira Videira
- **Ano:** 2001
- **Editora:** Centro Atlântico (Portugal)

Visão Geral

Esta obra aborda o ciclo completo da engenharia de software sob a ótica da modelagem orientada a objetos. Ela integra a especificação formal da UML com as principais metodologias de processo (como RUP - Rational Unified Process) e ensina a importância do uso de ferramentas CASE (Computer-Aided Software Engineering) para automatizar a engenharia reversa e a geração automática de código.

Conceitos-Chave

- **Processo de Desenvolvimento Unificado (RUP):** Foco nas fases clássicas de Concepção, Elaboração, Construção e Transição, aplicando modelagem iterativa e incremental.
- **Modelagem de Dados em UML (Data Modeling):** Como representar esquemas conceituais em diagramas de classes e realizar o mapeamento lógico e físico das tabelas para o banco de dados relacional SQL.
- **Model-Driven Engineering (MDE):** Introdução à engenharia dirigida por modelos, onde modelos de software são transformados automaticamente em código-fonte funcional por meio de ferramentas CASE de mercado (ex: Enterprise Architect, Rational).

6. Python e Orientação a Objetos (Curso PY-14)

Material Didático Institucional da Caelum

- **Autor:** Caelum (Ensino e Inovação)
- **Editora:** Apostila Oficial Caelum

[!TIP]

Relevância para a Disciplina:

É o ponto de partida técnico (Semanas 2, 3 e 4). Ajuda o estudante a traduzir os conceitos de orientação a objetos clássicos (geralmente ensinados em Java ou C++) para o ecossistema dinâmico e flexível do Python.

Visão Geral

A apostila ensina os fundamentos da linguagem de programação Python sob o paradigma da orientação a objetos, unindo noções básicas de lógica, manipulação de arquivos e tipos de dados com o desenvolvimento estruturado de classes.

Conceitos-Chave

- **Objetos e Classes em Python:** Compreensão dos conceitos de métodos construtores (`__init__`), referências de instância (`self`), herança simples, herança múltipla e polimorfismo no modelo de tipagem dinâmica do Python (duck typing).
- **Encapsulamento e Propriedades:** Uso das convenções de nomenclatura com prefixo de sublinhado simples/duplo (`_` / `__`) e o decorador `@property` para controle de acesso a atributos privados de maneira pythônica.
- **Tratamento de Exceções:** Como lançar (`raise`) e capturar (`try-except`) erros, o que é vital para a validação das regras de negócio do domínio.
- **Módulos e Organização de Projetos:** Como importar arquivos e organizar o projeto em pacotes contendo scripts separados para modularizar o código.

7. Tabela de Mapeamento de Conceitos e o Projeto Prático

Abaixo está o mapeamento dos conceitos abordados nas referências bibliográficas com as partes físicas da arquitetura que você precisa implementar ou utilizar na avaliação prática da disciplina (Semana 17 e 18):

| Arquivo da Avaliação | Padrão Arquitetural / Conceito Aplicado | Obra de Referência Principal | Descrição Prática no Projeto |

| :--- | :--- | :--- | :--- |

| **domain.py** | Domain Model, Entities, Value Objects, Aggregates, SOLID (SRP) | *Architecture Patterns with Python* (Cap. 1) & *SOLID para Ninjas* | Contém as classes que representam o negócio, suas regras de validação e restrições. Não deve possuir acoplamento com o banco de dados. |

| **repository.py** | Repository Pattern, Dependency Inversion (DIP) | *Architecture Patterns with Python* (Cap. 2) & *SOLID para Ninjas* | Classe abstrata e implementação concreta do repositório para acesso aos agregados do domínio no banco SQLite. |

| **unit_of_work.py** | Unit of Work Pattern (UOW), Context Manager | *Architecture Patterns with Python* (Cap. 6) | Gerencia a conexão com o banco de dados e controle de transação (commit/rollback) de forma atômica para cada caso de uso. |

| **messages.py** | Commands & Events (Message-Driven) | *Architecture Patterns with Python* (Cap. 7 & 8) | Definição das classes de mensagens (Comandos para ações imperativas, Eventos para reações a fatos ocorridos). |

| **handlers.py** | Service Layer, Command/Event Handlers | *Architecture Patterns with Python* (Cap. 4 & 8) | Funções ou classes que contêm os casos de uso que processam os Comandos/Eventos intermediados pelo Message Bus. |

| **messagebus.py** | Message Bus / Dispatcher | *Architecture Patterns with Python* (Cap. 8) | Despacha as mensagens recebidas do controlador da API para seus respectivos manipuladores em **handlers.py**. |

| **read_model.py** | CQRS (Command Query Responsibility Segregation) | *Architecture Patterns with Python* (Cap. 12) | Camada otimizada dedicada exclusivamente à consulta de dados (leitura rápida) sem passar pelas complexidades do modelo de escrita. |

| **test/** | Test-Driven Development (TDD) | *Architecture Patterns with Python* & *SOLID para Ninjas* | Testes de unidade e integração rápidos simulando as operações do negócio sem acoplamento de infraestrutura (usando fakes). |